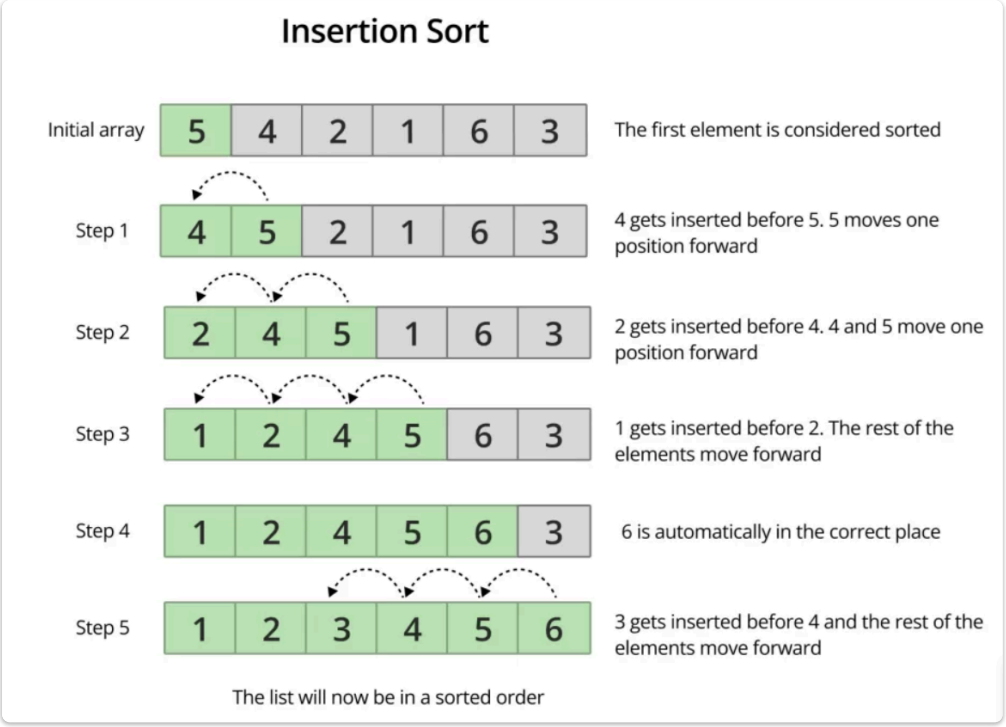
Insertion-Sort

Insertion Sort

Insertion Sort (ordenación por inserción) es un algoritmo de ordenación simple y estable que construye el array ordenado de manera incremental: recorre el array de izquierda a derecha y, en cada paso, inserta el elemento actual en la posición correcta dentro de la parte ya ordenada.

La analogía típica es ordenar una mano de cartas:

- Tienes en la mano algunas cartas ya ordenadas.
- Coges una carta nueva de la mesa.
- La insertas en el lugar adecuado entre las que ya tienes ordenadas.



Propiedades clave:

- In-place:
  No necesita estructuras auxiliares grandes, solo unas pocas variables.
- Estable:
  No cambia el orden relativo de elementos iguales.
- Complejidad temporal:
  - Mejor caso (array ya ordenado): (O(n))

    - Caso medio: (O(n^2))
    - Peor caso (orden inverso): (O(n^2))
- Uso típico:
    - Muy útil para listas pequeñas o casi ordenadas.
    - Se usa como "subrutina" en algoritmos híbridos (por ejemplo, cuando el tamaño del subarray es muy pequeño).

Esquema Insertion-Sort

Supongamos un array `A` de longitud `n`.

La idea es que:

- La posición `0` se considera subarray ordenado (un solo elemento siempre está "ordenado" respecto a sí mismo).
- Vamos avanzando con un índice `j` desde `1` hasta `n - 1`.
- En cada iteración, insertamos `A[ j ]` en su posición correcta dentro del subarray `A[0, ..., j-1]`, que ya está ordenado.

Paso a paso para una iteración genérica:

1. Elegir el elemento a insertar (la clave)
    - `key = A[j]`
    - Este es el elemento que vamos a "colocar" en la parte ordenada.
2. Empezar desde el elemento anterior
    - `i = j - 1`
    - `i` recorre la parte ordenada desde el final hacia el inicio (`i = j - 1, j - 2, ..., 0`).
3. Desplazar elementos mayores que la clave
    - Mientras:
        - no nos salgamos del array por la izquierda (`i >= 0`), y
        - el elemento en `A[i]` sea mayor que `key`,
    - hacemos:
        - `A[i + 1] = A[i]` → desplazamos `A[i]` una posición a la derecha.

    - `i = i - 1` → avanzamos hacia la izquierda para seguir comparando.
  - Esto va creando un "hueco" en la parte ordenada donde finalmente colocaremos la `key`.
4. **Insertar la clave en su hueco**
  - Cuando el bucle termina, hemos encontrado la posición correcta: `A[i + 1] = key`
  - En este punto, `A[0, ..., j]` queda ordenado.
5. **Repetir para el siguiente elemento**
  - Incrementamos `j` y repetimos los pasos 1–4 hasta llegar a `j = n - 1`.

Mini ejemplo rápido

Para `A = [5, 2, 4, 6, 1, 3]`:

- `j = 1`, `key = 2`, subarray ordenado inicial `[5]`
  → 2 < 5, desplazamos 5 a la derecha → `[5, 5, 4, 6, 1, 3]`
  → insertamos 2 en `A[0]` → `[2, 5, 4, 6, 1, 3]`
- `j = 2`, `key = 4`, subarray ordenado `[2, 5]`
  → 4 < 5, desplazamos 5 → `[2, 5, 5, 6, 1, 3]`
  → 4 ≥ 2, paramos y ponemos `key` en `A[1]` → `[2, 4, 5, 6, 1, 3]`

... y así hasta que todo el array queda ordenado.

Pseudocódigo Insertion-Sort

En CLRS el pseudocódigo original usa índices desde 1, pero aquí lo adaptamos a índices desde 0 (más cercano a lenguajes reales como Python/Java):

```
INSERTION-SORT(A)
    n = length(A)
    for j = 1 to n - 1
        key = A[j]
```

```
        i = j - 1
        while i >= 0 and A[i] > key
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

Implementación en Python (versión limpia)

```python
def insertion_sort(arr):
    # Recorremos desde el segundo elemento hasta el final
    for j in range(1, len(arr)):
        # Elemento que queremos insertar en la parte ordenada
        key = arr[j]
        i = j - 1

        # Desplazamos hacia la derecha los elementos mayores que key
        while i >= 0 and arr[i] > key:
            arr[i + 1] = arr[i]
            i -= 1

        # Insertamos key en su posición correcta
        arr[i + 1] = key


if __name__ == "__main__":
    unsorted_array = [3, -7, 10, -4, -2, 6, -1, 8, -9, 2]
    insertion_sort(unsorted_array)
    print(unsorted_array)  # [-9, -7, -4, -2, -1, 2, 3, 6, 8, 10]
```